

Synthetic Big Patent Similarity

Hajar Lamtaai

April 14, 2025

Contents

1	Introduction	2
2	Dataset	3
3	Task Design and Learning Objective	3
3.1	Initial Intuition and Alternative Directions	3
3.2	Task Formulation	3
4	Literature on Embedding Models for Semantic Textual Similarity	4
4.1	E5 (Embeddings from Evidence-based Examples)	4
4.2	MiniLM: Task-Aware Distillation of Transformers	5
4.3	SBERT: Siamese BERT for Sentence Similarity	5
5	Preliminary Results	5
5.1	Preprocessing	5
5.2	Model Description and Checkpoint	6
5.3	Zero-Shot Evaluation	6
5.4	Fine-Tuning Experiments with E5-base	6
6	Discussion	8
7	Next Steps	8

1 Introduction

This experiment is conducted in the context of the "Synthetic Big Patent Similarity" technical test. The task revolves around learning meaningful semantic representations of patent-related documents in order to measure document similarity.

This kind of task is key in the context of non-conformities (NC) management, especially in industrial domains where technical reports, incident logs, and corrective actions are documented in natural language. Managing large volumes of NCs is a recurring challenge, as we can tell from the dataset we are working on, reports vary in structure, terminology, and focus, making it difficult to retrieve relevant past issues or cluster similar problems. A model capable of embedding documents such that semantically similar cases are close together can serve as the backbone of smarter NC handling systems.

Scripts used are available at: github.com/hajar6883/representation-learning_test

2 Dataset

The dataset used is a curated subset of the BigPatent dataset, provided as

`dataset_big_patent_v1.json`

The dataset used in this experiment is composed of structured quadruples designed to support a contrastive learning task for semantic similarity. Each data sample consists of the following four elements:

- **Anchor (A)**: A paragraph extracted from a patent or technical document. These texts are often detailed and technically dense, describing inventions, systems, or problem contexts.
- **Query (Q)**: A natural language question that focuses the model’s attention on a specific semantic aspect of the anchor, such as its advantages, use cases, or operating principles.
- **Positive (P)**: A short and relevant answer or summary that aligns well with the query when applied to the anchor. It is semantically appropriate and contextually meaningful.
- **Negative (N)**: A distractor paragraph which may be topically similar to the anchor, but does not constitute a good response to the query. It serves to introduce contrast during training.

3 Task Design and Learning Objective

Given the structure of the dataset, this task naturally lends itself to a **similarity-based representation learning** framework. The goal is to learn embeddings that reflect the semantic relationship between documents, especially in the context of technical domains such as patents and industrial non-conformities.

3.1 Initial Intuition and Alternative Directions

An early intuition for tackling this problem—based on a survey of benchmark datasets and SOTA embedding models (e.g., MS-MARCO for query-passage retrieval)—was to approach it as a ranking passages in response to a query. Another idea was to *generate a synthetic response R from the anchor and query*, and then train an embedding model to enforce similarity between the response and the positive passage far pushing the negative one apart. However, this formulation shifted the task toward a **generative objective**, which diverged from the core objective outlined in the *Synthetic Big Patent Similarity* setup. The main challenge is not to answer questions or generate content, but rather to measure the semantic similarity between textual documents, guided by context when needed.

3.2 Task Formulation

A more appropriate approach involves designing an embedding model to capture contextual similarity by leveraging the query as a guide. Specifically, the model should learn embeddings such that:

$$\text{sim}(A, Q, P) > \text{sim}(A, Q, N)$$

This can be implemented by concatenating the anchor and query, and then comparing the resulting embedding to that of the positive and negative candidates. The objective is to ensure

that **semantically similar documents are embedded close together**, and dissimilar ones are far apart.

Formally, the model must:

- Map the **Anchor (A)**, **Positive (P)**, and **Negative (N)** into a shared embedding space.
- Enforce that $\text{sim}(A, P) > \text{sim}(A, N)$ using a metric such as cosine similarity.
- Utilize the **Query (Q)** to condition or guide the model’s notion of similarity (e.g., focusing on “advantages” or “applications”).

Two potential ways to incorporate the query into the training process were considered:

- **Input Augmentation:** Concatenate the query with the anchor or positive document during encoding to emphasize query-relevant aspects.
- **Query-Aware Loss:** Use a *weighted triplet loss* that emphasizes tokens or phrases in A and P that are most relevant to Q.
 - For instance, a language model could score each token’s alignment with the query focus (e.g., how much a word like `\compact` aligns with `\advantages`).
 - These scores can then be used to prioritize relevant content in the loss computation.

Another direction that was thought of also is a sort of “Query-Guided Attention”, the query can be encoded separately and used to create an attention vector over the anchor or positive embeddings, which is subject to helping the model attend more to query-relevant parts of the document, improving the alignment between A and P under the influence of Q.

Overall, the query serves as a semantic lens—helping the model focus on the most relevant dimensions of similarity during training.

4 Literature on Embedding Models for Semantic Textual Similarity

Recent advances in sentence and passage embeddings have significantly improved the ability of models to capture fine-grained semantic similarity across diverse domains. In this section, I briefly review three influential models that represent state-of-the-art techniques in this area and have informed the experimental choices: **E5**, **MiniLM**, and **SBERT**.

4.1 E5 (Embeddings from Evidence-based Examples)

We briefly present three prominent embedding models used for semantic similarity tasks, with an emphasis on their pretraining strategies, contrastive learning formulations, and fine-tuning mechanisms.

The E5 family of models are trained using large-scale web data and a weakly supervised contrastive learning objective. E5 treats tasks like retrieval and QA as a text-to-text matching problem and uses queries paired with relevant passages (and in some cases, irrelevant ones) to enforce the alignment between semantically related text. The model is trained using a dual-encoder architecture with contrastive loss defined as:

$$\mathcal{L}_{\text{cont}} = -\frac{1}{n} \sum_i \log \frac{e^{s_{\theta}(q_i, p_i)}}{e^{s_{\theta}(q_i, p_i)} + \sum_{j \neq i} e^{s_{\theta}(q_i, p_j)}} \quad (1)$$

where $s_\theta(q, p) = \cos(E_q, E_p)/\tau$ is the scaled cosine similarity between the embeddings of q and p , and τ is a temperature hyperparameter. Negative examples are drawn from other passages in the same batch.

For fine-tuning, E5 combines contrastive loss with distillation from a cross-encoder (CE) model. The loss becomes a weighted combination of KL divergence between soft teacher predictions and the student’s predictions, and the contrastive loss:

$$\min \mathcal{D}_{\text{KL}}(p_{\text{CE}}, p_{\text{student}}) + \alpha \mathcal{L}_{\text{cont}} \quad (2)$$

4.2 MiniLM: Task-Aware Distillation of Transformers

MiniLM is a lightweight yet highly competitive Transformer model based on BERT, distilled using a task-aware teacher-student approach. Rather than directly replicating token-level predictions, MiniLM leverages the *self-attention distributions and value matrices* of a larger teacher model (e.g., BERT or RoBERTa) to guide its training. This allows it to retain much of the semantic power of its larger counterparts while being highly efficient for deployment. When used in sentence embedding tasks (e.g., via the ‘all-MiniLM-L6-v2’ checkpoint), it delivers strong performance on benchmarks like STS (Semantic Textual Similarity) with very fast inference.

4.3 SBERT: Siamese BERT for Sentence Similarity

SBERT modifies the original BERT architecture by introducing a siamese or triplet network design, allowing it to generate fixed-size sentence embeddings suitable for direct similarity comparison using cosine similarity. Unlike BERT, which is not optimized for sentence-level tasks out of the box, SBERT is fine-tuned on Natural Language Inference (NLI) and semantic similarity datasets using a regression or contrastive objective, optimizing either:

- **Regression loss:** Mean Squared Error on similarity scores
- **Triplet loss:** For anchor a , positive p , and negative n :

$$\mathcal{L}_{\text{triplet}} = \max(0, \text{sim}(a, n) - \text{sim}(a, p) + \text{margin})$$

This allows it to capture semantic relationships between entire sentences or paragraphs, and it has become a foundational model in many retrieval and ranking pipelines.

These models all highlight the importance of **contrastive learning**, **domain diversity**, and **alignment between query and response pairs** in pretraining. Their performance on sentence-level semantic tasks makes them well-suited starting points for our contrastive learning experiments on technical documents such as patents or industrial non-compliance reports.

5 Preliminary Results

5.1 Preprocessing

While the `transformers` library and wrapper used for when using the models (`sentence_transformers`) abstract away tokenization and handles input formatting internally, we notice the considerable length of many input texts—particularly the **anchor** fields, which often exceed typical model limits (512 tokens). Moreover, since the data originates from OCR-processed documents, some residual noise and formatting inconsistencies were observed.

Considered strategies:

- **Truncation:** Cut the anchor at a fixed token limit.
- **Query-guided filtering:** Split the anchor into smaller semantically meaningful sections then select or emphasize anchor segments more relevant to the query (not yet implemented).

For the baseline, we decided to **pause complex preprocessing**. This is based on the assumption that the dataset (derived from another dataset) already includes some curation. Future experiments should revisit that..

5.2 Model Description and Checkpoint

We begin the experiment using the **intfloat/e5-base-v2** model, a transformer-based sentence encoder pre-trained for semantic retrieval tasks. The model is based on the BERT-base architecture and outputs 768-dimensional embeddings. The configuration includes 12 transformer layers, 12 attention heads, a hidden size of 768, and uses GELU activation with standard dropout settings. The pooling strategy relies on mean pooling over token embeddings, followed by normalization.

The model checkpoint was sourced from Hugging Face and was pre-trained using a *MultipleNegativesRankingLoss* objective across a large multi-task retrieval corpus. This training encourages the model to embed semantically similar sentences closer together, leveraging in-batch negatives rather than explicit negatives.

5.3 Zero-Shot Evaluation

We first evaluate the base model without any additional fine-tuning on our domain-specific triplet dataset. Each test sample consists of an *anchor*, a *positive* (semantically related), and a *negative* (less related). The model’s performance was measured using cosine similarity across the anchor–positive and anchor–negative pairs.

Table 1: Zero-Shot Cosine Accuracy and Model Characteristics on Triplet Evaluation Task (augmented anchors)

Model	Cosine Acc.	Archit.	Loss Obj.	Embedding Dim.
e5-base-v2	0.7000	BERT-base	MNLR	768
sentence-t5-base	0.7000	T5-base	Cosine Similarity	768
all-MiniLM-L6-v2	0.7100	DistilBERT	MNLR	384
bge-large-en	0.7200	RoBERTa-large	Contrastive	1024
e5-large	0.7400	BERT-large	MNLR	1024

5.4 Fine-Tuning Experiments with E5-base

To improve the model’s alignment with our task, we fine-tuned the **e5-base-v2** checkpoint using our labeled triplet dataset under three different training objectives: *TripletLoss*, *ContrastiveLoss*, and *MultipleNegativesRankingLoss* (MNRL).

TripletLoss

The **TripletLoss** leverages explicit (a, p, n) supervision, where a is the anchor, p is a semantically similar (positive) sentence, and n is a dissimilar (negative) sentence. The objective is to ensure that the anchor is closer to the positive than to the negative by at least a predefined margin δ :

$$\mathcal{L}_{\text{triplet}} = \max(0, \text{sim}(a, n) - \text{sim}(a, p) + \delta)$$

Despite its alignment with our labeled triplet format, this approach yielded a low cosine accuracy of **0.43**. We hypothesize that the default margin value $\delta = 1.0$ was overly strict given the semantic closeness between some negative examples, resulting in frequent violations and unstable training.

ContrastiveLoss

We reformulated our triplet dataset into binary sentence pairs (s_1, s_2) with a label $y \in \{0, 1\}$ indicating whether the two sentences are semantically similar. The **ContrastiveLoss** encourages similar pairs to have lower distances and dissimilar pairs to have distances greater than a margin δ :

$$\mathcal{L}_{\text{contrastive}} = y \cdot D^2 + (1 - y) \cdot \max(0, \delta - D)^2$$

where D is the distance between embeddings of s_1 and s_2 , typically using cosine or Euclidean distance. This formulation achieved a cosine accuracy of **0.96**, demonstrating its effectiveness. The binary structure provided a stable and expressive training signal, and the contrastive loss successfully leveraged both positive and negative labels.

MultipleNegativesRankingLoss (MNRL)

The **MNRL** objective is used during the original pretraining of E5. It relies only on (a, p) pairs and treats the other examples in the same batch as negatives. The goal is to maximize the similarity between anchor and positive, while minimizing it relative to all other in-batch sentences:

$$\mathcal{L}_{\text{mnrl}} = -\log \frac{\exp(\text{sim}(a, p))}{\sum_{i=1}^B \exp(\text{sim}(a, s_i))}$$

where s_i are the sentences in the batch (excluding p). Fine-tuning with this objective produced a cosine accuracy of **0.63**, outperforming TripletLoss but falling short of the contrastive setup. Its reliance on implicit negatives introduces noise, particularly in small or domain-specific datasets.

Table 2: Comparison of Training Objectives for Fine-Tuning on Triplet Evaluation Task

Training Setup	Loss Function	Cosine Accuracy
Zero-Shot (No Fine-Tuning)	None	0.700
Triplet Supervision	TripletLoss	0.430
Binary Pair Supervision	ContrastiveLoss	0.960
Positive Pairs Only	MultipleNegativesRankingLoss	0.630

6 Discussion

This experiment was not merely an exercise in applying pre-trained models to a new dataset, but an opportunity to interrogate the alignment between task formulation, training objective, and the nature of the data. A significant part of the process was dedicated to understanding what the task truly demanded like the ability to embed technical documents in a way that reflects their semantic similarity, guided by domain-specific queries.

In selecting a training objective, the initial assumption was that TripletLoss would be best suited to the task, as the dataset naturally lends itself to anchor–positive–negative triplet structure. However, the performance of this approach revealed important limitations. The default margin proved too rigid in practice, especially given the subtle semantic distinctions that exist between many technical negatives and positives. The model struggled to enforce the margin consistently, leading to lower overall accuracy.

In contrast, reformulating the task using a contrastive learning objective, treating pairs as explicitly similar or dissimilar—produced a dramatic improvement in performance. The simplicity of this framing allowed the model to focus on the core semantic distinction without the added complexity of margin-based ranking. Interestingly, although MultipleNegativesRankingLoss (used in the original E5 pretraining) was more robust than TripletLoss, it fell short of the contrastive approach. This suggests that in-batch negatives may not provide sufficient contrast in smaller or more homogeneous datasets.

One of the key takeaways from these experiments is that the structure of the dataset and the granularity of semantic variation it contains must inform the choice of learning objective. While triplet-style supervision may seem like the most direct translation of the data format, it is not always the most effective.

7 Next Steps

Following these findings, there are several promising directions for further investigation. First, refining the quality of negative examples is likely to enhance performance under margin-based objectives. Many negatives in the current dataset, while topically distinct, may not provide a strong enough semantic challenge to drive meaningful separation. Incorporating more sophisticated hard negative mining strategies or adversarial examples could strengthen contrastive supervision and enable more robust triplet-based learning.

Second, deeper integration of the query into the encoding process remains an open avenue. While initial experiments included simple concatenation of the query and anchor, more expressive architectures—such as using the query as an attention controller or conditioning mechanism—could help the model better align with the semantic intent behind each comparison.

Finally, while cosine accuracy on held-out triplets is a useful proxy, future experiments should evaluate the fine-tuned model on real-world tasks such as document retrieval, clustering, or anomaly detection within technical logs. These downstream evaluations will offer stronger validation of whether the learned embedding space supports the practical goals of non-conformity management in industrial settings.